CST1510 Programming for data communication and networks

Week 2: Control Structures (Selection & Iteration) + Lists

WORKSHOP

Objectives

- Control Statements
  - Selection
  - Iteration
- List

Algorithms

Any computing problem can be solved by executing a series of actions in a specific order. An algorithm is a procedure for solving a problem in terms of

- the actions to execute and
- the order in which these actions are executed

Specifying the order in which statements (actions) execute in a program is called program control.

Control Statements

- **Sequential Execution:** Statements execute in the order they are written.
- **Transfer of Control:** Python allows you to change the execution flow using control statements.

Types of Control

- **Sequential Execution:** Code runs step-by-step.
- **Selection (Conditional) Statements:** Directs the program to execute different code based on conditions.
- **Iteration (Looping) Statements:** Repeats code multiple times based on conditions.

No description has been provided for this image

Selection Statements

- **Three Types of Selection Statements:**
  - `if` **Statement:** Executes an action if a condition is `True`; skips the action if `False`.
  - `if…else` **Statement:** Executes one action if a condition is `True`, and a different action if `False`.
  - `if…elif…else` **Statement:** Chooses one of many actions based on multiple conditions.
- **Grouping Actions:** Wherever a single action is allowed, multiple actions can be used.

Iteration Statements

- **Two Types of Iteration Statements:**
  - `while` **Statement:** Repeats an action as long as a condition is `True`.
  - `for` **Statement:** Repeats an action for each item in a sequence.

`Selection Statements` intro

Boolean

A **Boolean** expression is an expression that results in a value, that is, either true or false. It is one of Python data type.

Decision Making: The if Statement and operators

- A **condition** is a Boolean expression with the value `True` or `False`.

In [1]:
```
10 > 5
```

Out[1]:
```
True
```

In [2]:
```
10 < 5
```

Out[2]:
```
False
```

`Comparison Operators`

Algebraic operator	Python operator	Sample condition	Meaning
>	`>`	`x > y`	`x` is greater than `y`
<	`<`	`x < y`	`x` is less than `y`
≥	`>=`	`x >= y`	`x` is greater than or equal to `y`
≤	`<=`	`x <= y`	`x` is less than or equal to `y`
=	`==`	`x == y`	`x` is equal to `y`
≠	`!=`	`x != y`	`x` is not equal to `y`

- Operators `>`, `<`, `>=` and `<=` have the same precedence.
- Operators `==` and `!=` have the same precedence, which is lower than `>`, `<`, `>=` and `<=`.

Examples:

In [3]:
```
a = 10
b = 5
```

In [4]:
```
print (f'{a} < {b} = ',a < b)
```

```
10 < 5 =  False
```

In [5]:
```
print (f'{a} <= {b} = ',a <= b)
```

```
10 <= 5 =  False
```

In [6]:
```
print (f'{a} > {b} = ',a > b)
```

```
10 > 5 =  True
```

In [7]:
```
print (f'{a} >= {b} = ',a >= b)
```

```
10 >= 5 =  True
```

In [8]:
```
print (f'{a} == {b} = ',a == b)
```

```
10 == 5 =  False
```

In [9]:
```
print (f'{a} != {b} = ',a !=b)
```

```
10 != 5 =  True
```

Confusing == and =

- Using the assignment symbol (`=`) instead of the equality operator (`==`) in an `if` statement's condition is a common syntax error.

In [10]:
```
a = 10
```

In [11]:
```
a == 10
```

Out[11]: True

Activity

What will be the result of the following statements:

In [ ]:
```
print('1 == 1', 1 == 1)
```

In [ ]:
```
print('1 <= 1', 1 <= 1)
```

In [ ]:
```
print ('1 != 2',1 != 2)
```

In [ ]:
```
print ('"good" != "bad"', "good" != "bad")
```

In [ ]:
```
print ('123 == "123"', 123 == "123")
```

`Logical operators`

Algebraic operator	Python operator	Sample condition	Meaning
∧	`and`	`x > y and y > z`	Both conditions are true
∨	`or`	`x > y or y > z`	At least one condition is true

- The `and` operator has a higher precedence than the `or` operator.
- Logical operators have lower precedence than comparison operators like `>`, `<`, `>=`, `<=`, `==`, and `!=`.

Boolean Operators `and`, `or` and `not`

Boolean Operator `and`

- Ensure that two conditions are both `True` with the **Boolean `and` operator**.

In [12]:
```
gender = 'Female'
```

In [13]:
```
age = 70
```

In [14]:
```
if gender == 'Female' and age >= 65:
    print('Senior female')
```

```
Senior female
```

`and`

Condition 1	Condition 2	Expression	Result
`True`	`True`	`True and True`	`True`
`True`	`False`	`True and False`	`False`
`False`	`True`	`False and True`	`False`
`False`	`False`	`False and False`	`False`

- Can write statements like this using a concise conditional expression.
- The parentheses are not required, but they make it clear that the statement assigns the conditional expression's value to `result`.

`and` Example

In [18]:
```
grade = 60
```

In [19]:
```
result = ('Passed' if grade >= 60 else 'Failed')
```

In [21]:
```
result
```

Out[21]:
```
'Passed'
```

In [22]:
```
# Python checks the first condition (False) and stops, because the whole
if 5 > 10 and print("This won't be printed"):
    print("This won't run")
```

Explanation: In an `and` expression, if the first condition is `False`, Python doesn't check the rest because the whole expression will be `False`. This improves performance by avoiding unnecessary checks

Boolean Operator `or`

- Ensure that one *or* both of two conditions are `True` with the **Boolean `or` operator**.

In [23]:
```
semester_average = 83
```

In [24]:
```
final_exam = 95
```

In [25]:
```
if semester_average >= 90 or final_exam >= 90:
    print('Student gets an A')
```

```
Student gets an A
```

`or`

Condition 1	Condition 2	Expression	Result
`True`	`True`	`True or True`	`True`
`True`	`False`	`True or False`	`True`
`False`	`True`	`False or True`	`True`
`False`	`False`	`False or False`	`False`

Short-Circuit Evaluation for Better Performance

- Python stops checking an `and` condition as soon as it finds `False`.
- It stops checking an `or` condition as soon as it finds `True`.
- For `and` conditions, place the most likely `False` check first.
- For `or` conditions, place the most likely `True` check first.

`or` Example

In [26]:
```
# Python checks the first condition (True) and stops, because the whole e
if 10 > 5 or print("This won't be printed"):
    print("This will run")
```

```
This will run
```

Explanation: In an `or` expression, if the first condition is `True`, Python stops and doesn't check the rest because the whole expression will be `True`.

Boolean Operator `not`

- "Reverse" the meaning of a condition.
- **Unary operator**—it has only *one* operand.

In [27]:
```
grade = 87
```

In [28]:
```
if not grade == -1:
    print('The next grade is', grade)
```

```
The next grade is 87
```

In [29]:
```
if grade != -1:
    print('The next grade is', grade)
```

```
The next grade is 87
```

- Truth table for the `not` operator.

ex	not ex
`False`	`True`
`True`	`False`

Activity:

What is the result of the following Boolean expressions?

In [ ]:
```
print ('(1 == 1 and 1 != 2) = ', 1 == 1 and 1 != 2)
```

In [ ]:
```
print('True and (3 > 4) ',True and (3 > 4))
```

In [ ]:
```
x = 1
print((x > 0) or (x < 0))
```

In [ ]:
```
(x != 0) or (x == 0)
```

In [ ]:
```

```

`Selection Statements`

`if` Statement

- Pseudocode: Suppose that a passing grade on an examination is 60. The pseudocode

  > If student's grade is greater than or equal to 60
  > Display 'Passed'

- If the condition is true, 'Passed' is displayed. Then, the next pseudocode statement in order is "performed."
- If the condition is false, nothing is displayed, and the next pseudocode statement is "performed."
- Indentation emphasizes that 'Passed' is displayed only if the condition is true.

In [33]:
```
grade = 85
```

In [34]:
```
if grade >= 60:
    print('Passed')
```

```
Passed
```

Indentations!!

In [35]:
```
if grade >= 60:
print('Passed')  # statement is not indented properly
```

```
Passed
```

In [ ]:
```
if grade >= 60:
    print('Passed')
  print('Good job!) # statement is not indented properly
```

Every Expression Can Be Treated as `True` or `False`

In [36]:
```
if 1:
    print('Nonzero values are true, so this will print')
```

```
Nonzero values are true, so this will print
```

In [37]:
```
if 0:
    print('Zero is false, so this will not print')
```

Activity:

- Create a variable age and assign it a value of 20.
- Use an if statement to print "Eligible to vote" if the age is 18 or above

In [ ]:
```

```

`if` ... `else` Statement Flowchart

- Sometimes the suites in an `if` ... `else` statement assign different values to a variable, based on a condition

In [38]:
```
if grade >= 60:
    print('Passed')
else:
    print('Failed')
```

```
Passed
```

- Assign `57` to `grade`, then shows the `if` ... `else` statement again to demonstrate that only the `else` suite executes

In [39]:
```
grade = 57
```

In [40]:
```
if grade >= 60:
    print('Passed')
else:
    print('Failed')
```

```
Failed
```

In [ ]:
```

```

In [41]:
```
grade = 99
```

In [42]:
```
if grade >= 60:
    print('Passed')
else:
    print('Failed')
```

```
Passed
```

In [ ]:
```

```

In [43]:
```
grade = 87
```

In [44]:
```
if grade >= 60:
    result = 'Passed'
else:
    result = 'Failed'
```

In [45]:
```
result
```

Out[45]:
```
'Passed'
```

Activity:

Check if a person has reached the legal drinking age.

- Assign 17 to the variable age.
- Use an if-else statement to print "Allowed to drink" if the age is 21 or above, and "Not allowed to drink" otherwise.

Multiple Statements

In [46]:
```
grade = 49
```

In [47]:
```
if grade >= 60:
    print('Passed')
else:
    print('Failed')
    print('You must take this course again')
```

```
Failed
You must take this course again
```

- If you do not indent the second `print`, then it's not in the `else`'s suite.
- In that case the statement always executes, creating strange incorrect output.

In [48]:
```
grade = 100
```

In [49]:
```
if grade >= 60:
    print('Passed')
else:
    print('Failed')
print('You must take this course again')
```

```
Passed
You must take this course again
```

`if`…`elif`…`else` Statement

- Can test for many cases.
- Only the action for the first `True` condition executes.

In [50]:
```
grade = 77
```

In [51]:
```
if grade >= 90:
    print('A')
elif grade >= 80:
    print('B')
elif grade >= 70:
    print('C')
elif grade >= 60:
    print('D')
else:
    print('F')
```

```
C
```

`else` Is Optional

- Handle values that do not satisfy any of the conditions.
- Without an `else`, if no conditions are `True`, the program does not execute any of the statement's suites.

Activity:

Determine if a student has passed or is at risk based on attendance.

- Assign 85 to the variable attendance.
- Print "Excellent attendance" if the attendance is above 90.
- Print "Good attendance" if the attendance is between 75 and 90 (inclusive).
- Print "At risk" if the attendance is below 75.

`Switch` Statement

> **Note:** Available only in Python 10 and above.

A switch statement allows you to execute one of several branches based on a comparison to a single value.

Example:

In [52]:
```
day = "Monday"
match day:
    case "Monday":
        print("Office Early")
    case "Tuesday":
        print("Office")
    case "Wednesday":
        print("Work From Home")
    case "Thursday":
        print("Gym then Office")
    case "Friday":
        print("Office Half Day")
```

```
    case _:
        print("Sleep In")
```

```
Office Early
```

In [ ]:

Activity

Days of the Week Activities Create a program that suggests an activity based on the day of the week.

- Assign the day of the week to the variable day.
- Use a match-case statement to suggest different activities for each day of the week.

Example activities:

```
- "Monday"- "Start your week with a morning jog!"
- "Tuesday"-"Attend the coding workshop."
- "Wednesday"-"Mid-week relaxation: Read a book."
- "Thursday"-"Time to hit the gym!"
- "Friday"- "Movie night with friends."
- "Saturday"-"Go hiking in nature."
- "Sunday"-"Family time and prepare for the week ahead."
- _ - "Invalid day.
```

In [ ]:

`Iteration Statements`

`while` Statement

- Repeats one or more actions while a condition remains `True`.

In [53]:
```
element = 3
```

In [54]:
```
while element <= 50:
    element = element * 3
```

In [55]:
```
element
```

Out[55]: 81

- To prevent an infinite loop, something in the `while` suite must change `product`'s value, so the condition eventually becomes `False`.

Activity:

Write a Python script to print the multiplication table of 3 using a while loop.

Task:

- Create a variable number and set it to 1.
- Use a while loop to print the product of 3 and number while number is less than or equal to 10.
- Increment number by 1 in each iteration.

`for` Statement

- Repeat an action or several actions for each item in a sequence of items.
- A string is a sequence of individual characters.

In [56]:
```
for char in 'cst1510':
    print(char, end=' ')
```

```
c s t 1 5 1 0
```

- When the `for` loop starts, Python assigns the first character of `'cst1510'` (i.e., `'c'`) to the **target variable** between `for` and `in`.
- After executing the loop body, Python assigns the next character (`'r'`) to the target variable and repeats the process.
- This continues until all items in the sequence have been processed.
- It's common to use the target variable in the loop body, but it's not mandatory.

The `end` Argument in the `print` Function

- By default, `print` displays its arguments and moves to the next line.
- You can change this behavior using the `end` argument:

  print(character, end=' ')

- `end` is a **keyword argument**, not an actual Python keyword.
- The Python Style Guide suggests no spaces around the `=` in keyword arguments.
- Keyword arguments are also known as named arguments.

Activity

Write a Python script to print each character of a string on the same line, separated by spaces.

- Create a variable course_code and set it to 'cst1510'.

- Use a for loop to iterate over each character in the string.
- Print each character on the same line, separated by a space.

In [ ]:

```

```

Iterables, Lists, and Iterators

- The sequence following `for` must be an **iterable**—an object that provides items one at a time.
- Lists are common iterables, defined as a series of items separated by commas and enclosed in square brackets (`[ ]`).

In [57]:

```
total = 0
```

In [58]:

```
for number in [2, -3, 0, 17, 9]:
    total = total + number
```

In [59]:

```
total
```

Out[59]:

```
25
```

- Every sequence has an associated iterator.
- The `for` statement uses this iterator to access each item in the sequence, one by one, until all items are processed.

Built-In `range` Function and Generators

- Creates an iterable representing a sequence of consecutive integers starting from 0 and ending before the specified value.

In [60]:

```
for counter in range(10):
    print(counter, end=' ')
```

```
0 1 2 3 4 5 6 7 8 9
```

- Function `range`'s two-argument version produces a sequence of consecutive integers from its first argument's value up to, but not including, the second argument's value

In [61]:

```
for number in range(5, 10):
    print(number, end=' ')
```

```
5 6 7 8 9
```

Function range's three-argument version produces a sequence of integers from its first argument's value up to, but not including, the second argument's value, incrementing by the third argument's value (the step)

In [62]:
```
for number in range(0, 10, 2):
    print(number, end=' ')
```

```
0 2 4 6 8
```

If the third argument is negative, the sequence progresses from the first argument's value down to, but not including the second argument's value, decrementing by the third argument's value

In [63]:
```
for number in range(10, 0, -2):
    print(number, end=' ')
```

```
10 8 6 4 2
```

In [ ]:
```

```

Activity:

Counting Study Hours Scenario: You have planned to study for 5 hours in a day. Write a Python script to print a message for each hour completed.

- Use a for loop with range to print a message for each hour from 1 to 5. (You dont have to use actual time but you can if you like. (import time and use time.speep() to pose))

Activity:

Counting Down Days to an Event Scenario: You have an important event (e.g., a final exam or a concert) in 7 days. Create a countdown.

- Use a for loop with range to print the countdown from 7 to 1.

Augmented Assignments

- Simplify assignment expressions where the same variable appears on both sides of the `=`.

In [64]:
```
total = 0
```

In [65]:
```
for number in [1, 2, 3, 4, 5]:
    total += number  # add number to total and store in number
```

In [66]:
```
total
```

Out[66]:
```
15
```

- In the following table, assume: `a = 6`, `b = 10`, `x = 15`, `y = 4`, `z = 18`, `w = 7`

| Augmented Assignment | Sample Expression | Explanation | Assigns | :----------:|:----------:|:----------:|:-----:| `+=` | `a += 5` | `a = a + 5` | `11 to a` | `-=` | `b -= 3` | `b = b - 3` | `7 to b` | `*=` | `x *= 2` | `x = x * 2` | `30 to x` | `**=` | `y **= 2` | `y = y ** 2` | `16 to y` | `/=` | `z /= 3` | `z = z / 3` | `6.0 to z` | `//=` | `z //= 4` | `z = z // 4` | `4 to z` | `%=` | `w %= 5` | `w = w % 5` | `2 to w`

Activity:

```
Calculate the total sum of the numbers in a list using
augmented assignment.
```

- Create a variable total and set it to 0.
- Use a for loop to iterate through the list [10, 20, 30, 40, 50].
- Use the += operator to add each number to total.
- Print the final value of total.

Sentinel-Controlled Iteration

- Uses a special **sentinel value** to signal the end of data entry.
- Also known as **indefinite iteration** because the number of iterations is unknown beforehand.
- The sentinel value must be distinct from valid inputs to avoid confusion.

In [67]:
```
# Example

again = 'y'
while again =='y':
    print('Hello')
    again=input('Do you want to loop again? ')
```

```
Hello
Do you want to loop again? y
Hello
Do you want to loop again? n
```

In [68]:
```
# Example

flag = 1
while flag != '0':

    flag = input('To finish enter 0 > ')
```

```
To finish enter 0 > 1
To finish enter 0 > 0
```

Activity:

To practice using sentinel-controlled iteration to keep adding numbers until the user enters the sentinel value -1 to stop.

Instructions Task:

- Create a Python script that continuously prompts the user to enter numbers.
- Use a sentinel value of -1 to stop the loop.
- Keep a running total of all numbers entered (excluding -1).
- After the loop ends, print the total sum of the numbers entered.

`break` and `continue` Statements

- A `break` statement instantly terminates a `while` or `for` loop.

In [69]:
```
for number in range(100):
    if number == 10:
        break
    print(number, end=' ')
```

```
0 1 2 3 4 5 6 7 8 9
```

- A `continue` statement skips the rest of the current loop iteration.
  - In a `while` loop, it rechecks the condition before continuing.
  - In a `for` loop, it moves to the next item in the sequence (if available).

In [70]:
```
for number in range(10):
    if number == 5:
        continue
    print(number, end=' ')
```

```
0 1 2 3 4 6 7 8 9
```

In [ ]:
```

```

Activity:

Add statment to the code below to exit the loop

In [ ]:
```
while True:
    choice = input('Enter;\n 1-addition\n 2-subtraction\n',
                   '3-multiplication\n 0-Exit\n >')
    if choice == '1':
        print('Addition')
    elif choice == '2':
        print('Subtraction')
```

Activity:

Write a Python script that prints numbers from 1 to 10, but skips printing the number 5.

- Use a for loop to iterate through numbers from 1 to 10.
- Use a continue statement to skip printing the number 5.

Importing Modules

A module in Python is an additional library that provides extra functionality.

To use a module's functions, you need to import it using the **import** statement.

An import statement includes the `import` keyword followed by the module name, typically placed at the top of your Python file.

> **Note:** Some modules need to be installed separately, while others are included with Python.

Generating Random Numbers

Use the `random` module to generate random numbers.

- `random.randint(a, b)` : Returns a random integer between `a` and `b` (inclusive).
- `random.random()` : Returns a random float between `0` and `1`.

In [71]:
```
import random

random.random()   # return number between 0-1

#random.randint (min, max)
print(random.randint(1,5)) # return number between 1,5
```

```
2
```

In [ ]:
```

```

LISTs

List items are **ordered**, **changeable**, and **allow duplicate** values.

- **Ordered** means that the items have a defined order and that order will not change. If you add new items to a list, the new items will be placed at the end of the list. (Note: Some list methods will change the order, but in general, the order of the items will not change)
- **Changeable**: We can change, add, and remove items from a list after creation.
- **Allow duplicate** values: lists can have items with the same value.

In [72]:
```
l = [-4, 1, 0, 72, 1543]
```

In [73]:
```
print(l)
```

```
[-4, 1, 0, 72, 1543]
```

Creating Lists

The list class defines lists. To create a list, you can use the list's constructor as follows:

In [74]:
```
list0 = list()
```

In [75]:
```
list1 = list([2, 3, 4])
```

In [76]:
```
list3 = list(range(2, 6))
```

In [77]:
```
list4 = list("abcd")
```

In [78]:
```
list01 = []
```

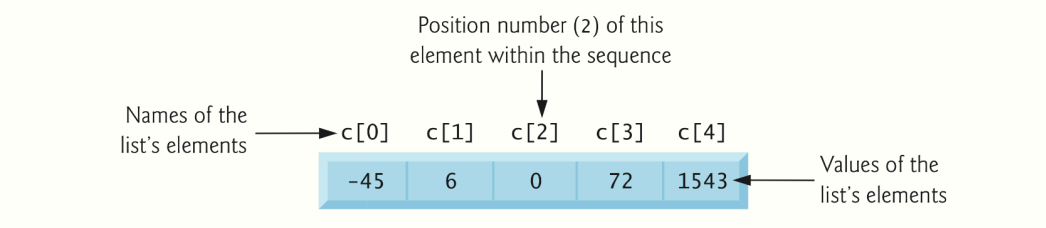
In [79]:
```
list02 = [2, 3, 4]
```

In [80]:
```
list03 = ["red", "green"]
```

In [81]:
```
list04 = [2, "three", 4, True, 2, 12.5]
```

Accessing List Elements

- Access a list element by writing the list's name followed by the element's **index** inside `[]`.



In [82]:
```
print(l) # the whole list
```

```
[-4, 1, 0, 72, 1543]
```

In [83]:
```
l[0]
```

Out[83]:
```
-4
```

In [84]:
```
l[4]
```

Out[84]:
```
1543
```

In [ ]:

Activity

- Create a list called friends containing the names of at least five of your friends as strings.

In [ ]:

```

```

Determining a List's Length

In [85]:

```
len(l)
```

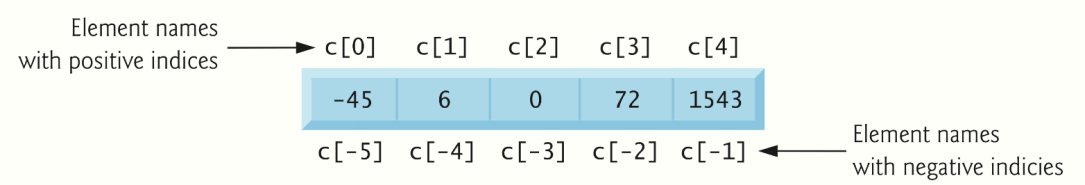
Out[85]: 5

Activity:

- Use the len() function to find out how many friends are in the list friends. Print the number of friends.

Accessing Elements from the End of the List with Negative Indices

- Lists can be accessed from the end by using *negative indices*:



In [86]:

```
l[-1]
```

Out[86]: 1543

In [87]:

```
l[-5]
```

Out[87]: -4

Activity

- Print the last and second-last friend in the list using negative indices -1 and -2.

Accessing all elements from the list

In [88]:

```
print(l)
```

```
[-4, 1, 0, 72, 1543]
```

In [89]:

```
# Iterate over each element in the list 'l'
for i in l:
```

```
    print(i) # # Print the current element 'i'
```

```
-4
1
0
72
1543
```

In [90]:
```
# Iterate over the indices of the list 'l'
for i in range(len(l)):
    print(l[i]) # # Print the element at index 'i' of the list 'l'
```

```
-4
1
0
72
1543
```

Activity

- Print each friend's name along with a number in front of it using a for loop.

List Unpacking

In [91]:
```
numbers = [10, 20, 30]
x, y, z = numbers  # Unpack the list into individual variables

print(f"X: {x}, Y: {y}, Z:{z}")
```

```
X: 10, Y: 20, Z:30
```

Lists Are Mutable

In [92]:
```
l
```

Out[92]:
```
[-4, 1, 0, 72, 1543]
```

In [93]:
```
l[4] = 17
```

In [94]:
```
l
```

Out[94]:
```
[-4, 1, 0, 72, 17]
```

Some Sequences Are Immutable

- Python's string and tuple sequences are immutable.

In [95]:
```
l1 = 'hello'
```

In [96]:
```
l1[0]
```

Out[96]:
```
'h'
```

In [97]:
```
l1[0] = 'H'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[97], line 1
----> 1 l1[0] = 'H'

TypeError: 'str' object does not support item assignment
```

Built-in functions on list

Add elements into a list

The **append()** method allow adding one element at the time

In [98]:
```
l1 = [2, 'three', 4, True, 2, 12.5]
```

In [99]:
```
l1
```

Out[99]:
```
[2, 'three', 4, True, 2, 12.5]
```

- The append() method adds a single element to the end of a list.

In [100…
```
l1.append(10)
```

In [101…
```
l1
```

Out[101…
```
[2, 'three', 4, True, 2, 12.5, 10]
```

- The extend() method adds multiple elements to a list.

In [102…
```
l1.extend([1,2])
```

In [103…
```
l1
```

Out[103…
```
[2, 'three', 4, True, 2, 12.5, 10, 1, 2]
```

Activity

- Add a new friend "Frank" to the list using the append() method.
- Extend the list with two more friends ["Grace", "Hank"] using the extend() method.
- Print the updated list.

Update list

- The insert() method inserts an item at the specified index:

In [104…
```
l1.insert(2, 'CST')
```

In [105…
```
l1
```

Out[105…
```
[2, 'three', 'CST', 4, True, 2, 12.5, 10, 1, 2]
```

Activity

- Insert a friend named "Ivy" at the third position in the list using the insert() method. Print the updated list.

Remove/Delete

- The **del** keyword remove an element from the list by index

In [106…
```
del l1[0]
```

In [107…
```
l1
```

Out[107…
```
['three', 'CST', 4, True, 2, 12.5, 10, 1, 2]
```

- The **remove()** method removes the specified element by its value

In [108…
```
l1.remove('CST')
```

In [109…
```
l1
```

Out[109…
```
['three', 4, True, 2, 12.5, 10, 1, 2]
```

- The **pop()** method removes the last element from a list when no index number is pass on as an attribute.

In [110…
```
l1.pop()
```

Out[110…
```
2
```

In [111…
```
l1
```

Out[111…
```
['three', 4, True, 2, 12.5, 10, 1]
```

Activity:

- Remove the first friend from the list using the del keyword.
- Remove the friend "Eve" using the remove() method.
- Print the updated list.

- The **clear()** removing all elements from list

In [112…
```
l1.clear()
```

In [113…
```
l1
```

Out[113…
```
[]
```

In [ ]:

In [ ]:

More functions and methods on list

The **index()** returns the first appearance of the specified value.

In [114…
```
months = ['January', 'February', 'March']
```

In [115…
```
print(months.index('February'))
```

```
1
```

Activity:

- Find and print the index of the friend "Charlie" using the index() method.

- The **min()** function returns the smallest element in sequence

In [116…
```
numbers = [5,3,2,7,20,106,2,2]
```

In [117…
```
print(min(numbers))
```

```
2
```

- The **max()** function returns the largest element in sequence

In [118…
```
print(max(numbers))
```

```
106
```

- The **sum()** function sum all numbers in sequence

In [119…
```
print(sum(numbers))
```

```
147
```

- The **count()** function returns the number of times element x appears in the list

In [31]:
```
print((numbers.count(2)))
```

```
3
```

Activity

new_friends = ['Alice', 'Bob', 'Charlie', 'David', 'Alice', 'Eve', 'Bob']

- Use the count() function to find out how many times the name "Alice" appears in the new_friends list.
- Print the result in a sentence format.

- The **sort()** function sorts the elements in the list in ascending order

In [120…
```
numbers = [5, 3, 2, 7, 20, 106, 2, 1, 2]
```

In [121…
```
numbers.sort()
```

In [122…
```
numbers
```

Out[122…
```
[1, 2, 2, 2, 3, 5, 7, 20, 106]
```

- The **sort()** function can sort a list in descending order with the attribute **'reverse=True'**

In [123…
```
numbers.sort(reverse=True)
```

In [124…
```
numbers
```

Out[124…
```
[106, 20, 7, 5, 3, 2, 2, 2, 1]
```

- The **reverse()** function reverses the elements in the list

In [125…
```
numbers.reverse()
```

In [126…
```
numbers
```

Out[126…
```
[1, 2, 2, 2, 3, 5, 7, 20, 106]
```

Activity:

- Sort the list in alphabetical order using the sort() method.
- Print the sorted list.

- The **shuffle()** function shuffled the elements in list1

In [127…
```
import random
random.shuffle(numbers)
```

In [128…
```
numbers
```

Out[128…
```
[2, 1, 2, 20, 7, 106, 3, 2, 5]
```

The **map() and zip()** functions are quite advanced but very interesting and useful

- The **map()** function executes a specified function for each item in an iterable. The item is sent to the function as a parameter.

In [129…
```
def myfunc(n):
    return len(n)
```

In [130…
```
words_size = map(myfunc, ['apple', 'banana', 'cherry'])
```

In [131…
```
print(words_size)
```

```
<map object at 0x111a3bc40>
```

In [132…
```
print(list(words_size))
```

```
[5, 6, 6]
```

- The **zip()** groups together elements from two or more lists

In [133…
```
months = ['January', 'February', 'March']
```

In [134…
```
days = [1,1,10]
```

In [135…
```
days_in_mointh = zip(months, days)
```

In [136…
```
print(days_in_mointh)
```

```
<zip object at 0x111f24240>
```

In [137…
```
print(list(days_in_mointh))
```

```
[('January', 1), ('February', 1), ('March', 10)]
```

Merging lists

To merge two lists we can use + operator to merge/concatenate two or more lists.

In [138…
```
list1=[1, 2, 5, 10]
list2=[5, 8, 20]

new_list = list1 + list2

new_list
```

Out[138…
```
[1, 2, 5, 10, 5, 8, 20]
```

Common membership operator

- The **in** used to check if an element is found in the list

In [139…
```
s = [1,3,5,7,9,10,20,35]
x = 5
print(x in s)
```

```
True
```

In [140…
```
numbers = [1,3,5,7,9,10,20,35]
x = 5
print(x in numbers)
```

```
True
```

- The **not in** used to check if an element does not present in the list

In [46]:
```
s = [1,3,5,7,9,10,20,35]

print(5 not in s)
```

```
False
```

Relational/ Comparison Operators on lists

You can compare lists using the comparison operators (>, >=, <, <=, ==, and !=). For this to work, the two lists must contain the same type of elements. The comparison uses lexicographical ordering: the first two elements are compared, and if they differ this deter- mines the outcome of the comparison; if they are equal, the next two elements are compared, and so on, until either list is exhausted.

In [141…
```
print([1,2,3] == [1,2,3])
```

```
True
```

In [142…
```
print([3,33,3] > [1,2,3])
```

```
True
```

In [143…
```
print([3,33,3] < [1,2,3])
```

```
False
```

In [144…
```
print([3,33,3] != [1,2,3])
```

```
True
```

In [ ]: